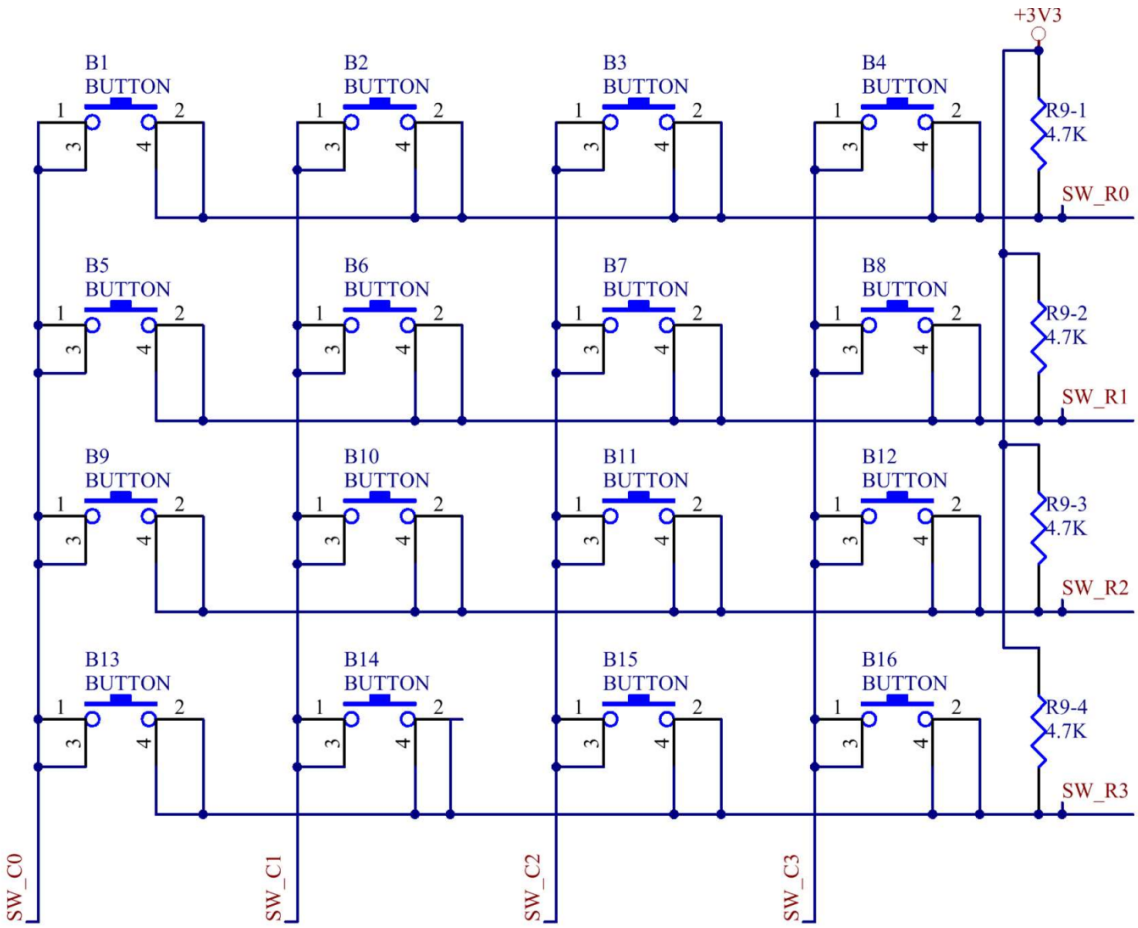
实验4实验报告

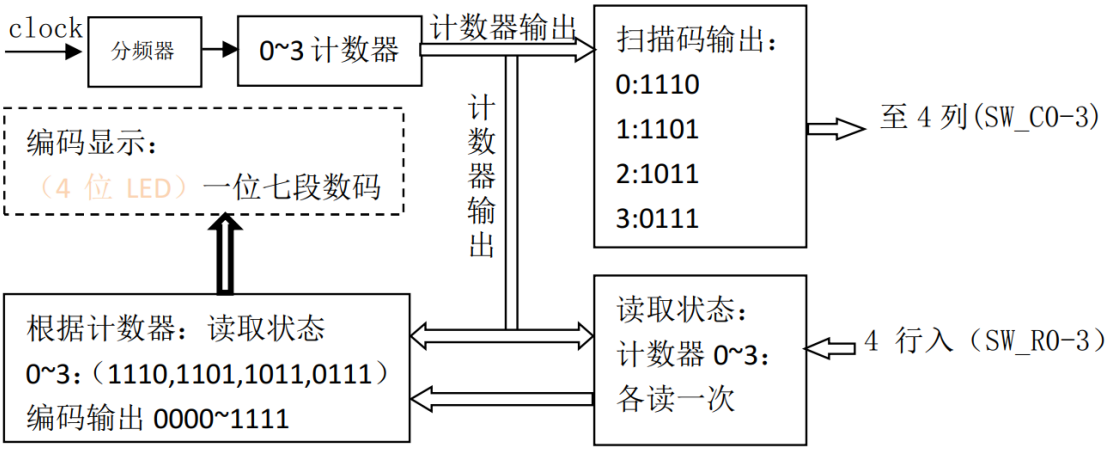
李毅PB22051031

1.实验设计

矩阵键盘是一个由 4×4 的按键开关组成的阵列，可实现 16 种状态的输入。4×4 矩阵键盘的硬件结构如图 1 所示，其中“+3V3”为 3.3V 的正电源，BUTTON按键，R 为电阻。SW_R0~SW_R3（简写为 R0~R3）为按键阵列连接到 FPGA 的行控制信号，SW_C0~SW_C3（简写为 C0~C3）为按键阵列连接到 FPGA 的列控制信号。



四个列控制信号 C0~C3 循环输出“1110、1101、1011、0111”来驱动键盘阵列，每输出一个列序列，紧接着就读取相应的 4 个行信号。通过读取的数据或状态来判断 16 个按键中有没有按键被按下、哪个键被按下，并对其状态做编码输出。此电路不停的工作，以便实时准确地获取键盘的输入状态，以供其它电路使用，从而实现了键盘阵列的扫描输入。扫描控制电路框图如下：



2.代码

```
`timescale 10ns/1ns

module FPGA_EXP4 (
    input clk,
    input rst,
    input [3:0] row_signal,
    output [3:0] col_signal,
    output [6:0] LED_data,
    output LED_sel
);
wire clk2;
//分频器
div50MHZ div(
    .clk(clk),
    .clkout(clk2),
    .rst(rst)
);
//矩阵扫描键盘电路
FPGA_EXP4_core fpga(
    .clk(clk2),
    .rst(rst),
    .row_signal(row_signal),
    .col_signal(col_signal),
    .LED_data(LED_data),
    .LED_sel(LED_sel)
);

endmodule

module FPGA_EXP4_core (
    input clk,
    input rst,
    input [3:0] row_signal,
    output reg [3:0] col_signal,
    output [6:0] LED_data,
    output LED_sel
);
reg [1:0] st;
reg [3:0] position;
wire [3:0] data;
assign LED_sel=1;
```

```
//列控制信号的产生
always @(posedge clk or negedge rst) begin
    if(!rst) begin
        st<=0;
    end
    else begin
        case (st)
            2'b00:begin
                st<=2'b01;
                col_signal<=4'b1110;
            end
            2'b01:begin
                st<=2'b10;
                col_signal<=4'b1101;
            end
            2'b10:begin
                st<=2'b11;
                col_signal<=4'b1011;
            end
            2'b11:begin
                st<=2'b00;
                col_signal<=4'b0111;
            end
            default: begin
                st<=2'b00;
            end
        endcase
    end
end
//确定按下的按键
always @(negedge clk or negedge rst) begin
    if(!rst) begin
        position<=4'b1101;
    end
    else begin
        case (row_signal)
        4'b1110: begin
            position[3:2]<=2'b00;
            case (col_signal)
                4'b1110: position[1:0]<=2'b00;
                4'b1101: position[1:0]<=2'b01;
                4'b1011: position[1:0]<=2'b10;
                4'b0111: position[1:0]<=2'b11;
                4'b1111: position[1:0]<=position[1:0];
                default: position[1:0]<=2'b00;
            endcase
        end
        4'b1101: begin
            position[3:2]<=2'b01;
            case (col_signal)
                4'b1110: position[1:0]<=2'b00;
                4'b1101: position[1:0]<=2'b01;
                4'b1011: position[1:0]<=2'b10;
                4'b0111: position[1:0]<=2'b11;
                4'b1111: position[1:0]<=position[1:0];
                default: position[1:0]<=2'b00;
            endcase
        end
```

```
        4'b1011: begin
            position[3:2]<=2'b10;
            case (col_signal)
                4'b1110: position[1:0]<=2'b00;
                4'b1101: position[1:0]<=2'b01;
                4'b1011: position[1:0]<=2'b10;
                4'b0111: position[1:0]<=2'b11;
                4'b1111: position[1:0]<=position[1:0];
                default: position[1:0]<=2'b00;
            endcase
        end
        4'b0111: begin
            position[3:2]<=2'b11;
            case (col_signal)
                4'b1110: position[1:0]<=2'b00;
                4'b1101: position[1:0]<=2'b01;
                4'b1011: position[1:0]<=2'b10;
                4'b0111: position[1:0]<=2'b11;
                4'b1111: position[1:0]<=position[1:0];
                default: position[1:0]<=2'b00;
            endcase
        end
        4'b1111: position<=position;
        default: position<=4'b1101;
        endcase
    end
end
//位置转换为位置信息
position_data pos_data(
    .position(position),
    .data(data)
);
//4-7译码器
LED_decoder decoder(
    .in(data),
    .out(LED_data)
);

endmodule

module position_data (
    input [3:0] position,
    output reg [3:0] data
);
    always @(position) begin
        case (position)
            4'b0000:data<=4'b0001;//1
            4'b0001:data<=4'b0010;//2
            4'b0010:data<=4'b0011;//3
            4'b0011:data<=4'b1010;//A(10)
            4'b0100:data<=4'b0100;//4
            4'b0101:data<=4'b0101;//5
            4'b0110:data<=4'b0110;//6
            4'b0111:data<=4'b1011;//B(11)
            4'b1000:data<=4'b0111;//7
            4'b1001:data<=4'b1000;//8
            4'b1010:data<=4'b1001;//9
            4'b1011:data<=4'b1100;//C(12)
```

```
            4'b1100:data<=4'b0000;//*
            4'b1101:data<=4'b0000;//0
            4'b1110:data<=4'b0000;//#
            4'b1111:data<=4'b1101;//D(13)
            default:data<=4'b0000;//0
        endcase
    end
endmodule

module LED_decoder(
    input [3:0] in,
    output reg [6:0] out
);
    always @(*) begin
        case (in)
            4'b0000: out = 7'b0000001;
            4'b0001: out = 7'b1001111;
            4'b0010: out = 7'b0010010;
            4'b0011: out = 7'b0000110;
            4'b0100: out = 7'b1001100;
            4'b0101: out = 7'b0100100;
            4'b0110: out = 7'b0100000;
            4'b0111: out = 7'b0001111;
            4'b1000: out = 7'b0000000;
            4'b1001: out = 7'b0000100;
            4'b1010: out = 7'b0001000;
            4'b1011: out = 7'b1100000;
            4'b1100: out = 7'b0110001;
            4'b1101: out = 7'b1000010;
            4'b1110: out = 7'b0110000;
            4'b1111: out = 7'b0111000;
        endcase
    end
endmodule

module div50MHZ(
    input clk,
    input rst,
    output reg clkout
);
    integer cnt;
    always @(posedge clk or negedge rst) begin
        if(!rst)
        begin
            cnt <= 0;
            clkout <= 1'b0;
        end
        else
        begin
            if(cnt <= 24999 && cnt>= 0)
                begin
                    clkout <= 1'b1;
                    cnt <= cnt + 1;
                end
            else if(cnt >=25000 && cnt <= 49999)
                begin
                    clkout <= 1'b0;
                    cnt <= cnt + 1;
```

```
                end
            else
                begin
                    cnt<=0;
                end
        end

    end
endmodule
```

3.实验结果

3.1 仿真波形

注：仿真时为了方便查看波形，未接分频器

激励文件：

```
`timescale 10ns/1ns

module FPGA_EXP4_tb();
reg         clk;
reg         rst;
reg   [3:0] row_signal;
wire  [3:0] col_signal;
wire  [6:0] LED_data;

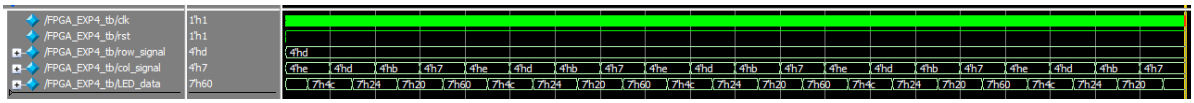
FPGA_EXP4 fpga(
    .clk(clk),
    .rst(rst),
    .row_signal(row_signal),
    .col_signal(col_signal),
    .LED_data(LED_data)
);

initial begin
    rst=0;
    #5 rst=1;
    #5 row_signal=4'b1110; //行信号输入
    #40 row_signal=4'b1101;
    #40 row_signal=4'b1011;
    #40 row_signal=4'b0111;
    #40 row_signal=4'b1101;
end

initial clk=0;
always #5 clk = ~clk; //产生时钟信号

endmodule
```

仿真波形

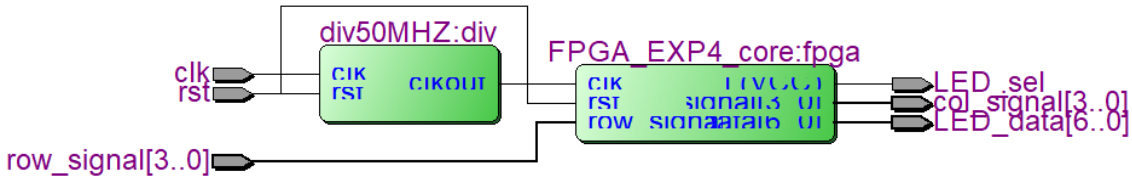


3.2 RTL电路

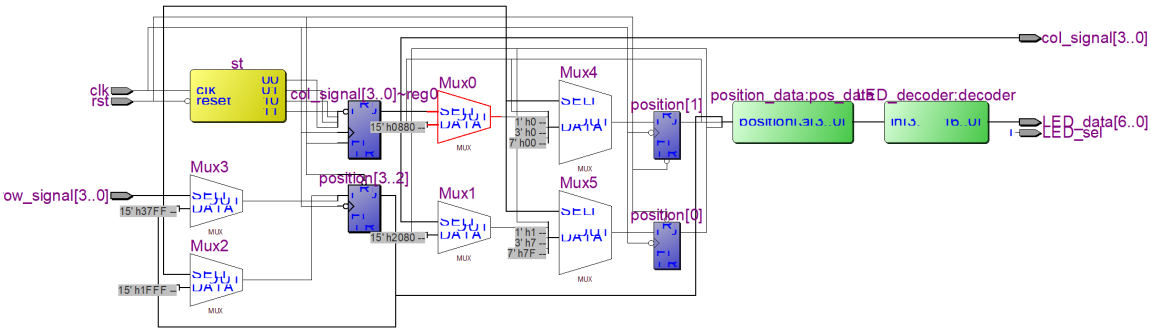
全编译之后的资源占用信息：

Flow Summary	
Flow Status	Successful - Mon Nov 04 14:42:26 2024
Quartus II 64-Bit Version	13.0.1 Build 232 06/12...3 SP 1 SJ Full Version
Revision Name	FPGA_EXP4
Top-level Entity Name	FPGA_EXP4
Family	Cyclone V
Device	5CEFA2F23C8
Timing Models	Final
Logic utilization (in ALMs)	40 / 9,430 (< 1 %)
Total registers	50
Total pins	18 / 224 (8 %)
Total virtual pins	0
Total block memory bits	0 / 1,802,240 (0 %)
Total DSP Blocks	0 / 25 (0 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI PMA RX ATT Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total HSSI PMA TX ATT Serializers	0
Total PLLs	0 / 4 (0 %)
Total DLLs	0 / 4 (0 %)

RTL电路：



其中：FPGA_EXP4_core:fpga RTL电路如下所示：



3.3 上板结果

管脚分配：

Node Name	Direction	Location	:/O Bank	REF Grou	er Locat	Reserved	) Standa	ent Stre	lew Rat	e
LED_data[6]	Output	PIN_K19	7A	B7A_N0	PIN_K19		2.5...lt)	12m...lt)	1 (...lt)	
LED_data[5]	Output	PIN_H18	7A	B7A_N0	PIN_H18		2.5...lt)	12m...lt)	1 (...lt)	
LED_data[4]	Output	PIN_K20	7A	B7A_N0	PIN_K20		2.5...lt)	12m...lt)	1 (...lt)	
LED_data[3]	Output	PIN_M18	5B	B5B_N0	PI...18		2.5...lt)	12m...lt)	1 (...lt)	
LED_data[2]	Output	PIN_E16	7A	B7A_N0	PIN_E16		2.5...lt)	12m...lt)	1 (...lt)	
LED_data[1]	Output	PIN_G13	7A	B7A_N0	PIN_G13		2.5...lt)	12m...lt)	1 (...lt)	
LED_data[0]	Output	PIN_G17	7A	B7A_N0	PIN_G17		2.5...lt)	12m...lt)	1 (...lt)	
LED_sel	Output	PIN_C11	7A	B7A_N0	PIN_C11		2.5...lt)	12m...lt)	1 (...lt)	
clk	Input	PIN_W16	4A	B4A_N0	PI...16		2.5...lt)	12m...lt)		
col_signal[3]	Output	PIN_AB22	4A	B4A_N0	PI...22		2.5...lt)	12m...lt)	1 (...lt)	
col_signal[2]	Output	PIN_AB20	4A	B4A_N0	PI...20		2.5...lt)	12m...lt)	1 (...lt)	
col_signal[1]	Output	PIN_AA20	4A	B4A_N0	PI...20		2.5...lt)	12m...lt)	1 (...lt)	
col_signal[0]	Output	PIN_Y17	4A	B4A_N0	PIN_Y17		2.5...lt)	12m...lt)	1 (...lt)	
row_signal[3]	Input	PIN_V21	4A	B4A_N0	PIN_V21		2.5...lt)	12m...lt)		
row_signal[2]	Input	PIN_W21	4A	B4A_N0	PI...21		2.5...lt)	12m...lt)		
row_signal[1]	Input	PIN_Y19	4A	B4A_N0	PIN_Y19		2.5...lt)	12m...lt)		
row_signal[0]	Input	PIN_Y21	4A	B4A_N0	PIN_Y21		2.5...lt)	12m...lt)		
rst	Input	PIN_Y11	3B	B3B_N0	PIN_Y11		2.5...lt)	12m...lt)		
<<new node>>										

上板结果：

按下按键，数码管可以正常显示，验收通过

4.实验总结

本次实验对verilog语言，和矩阵键盘扫描电路有了更全面认识。